

OAI — Adding a Custom UE

Runbook & Troubleshooting Log

A precise record of adding a second subscriber (208990100001189) to the running 5G core, getting it to register, and establishing **UE-to-UE communication**. Three parts: **(1) commands in order**, **(2) every problem we hit with its cause and fix**, and **(3) how the two UEs were made to talk**.

- Deployment dir: ci-scripts/yaml_files/5g_rfsimulator/
- New UE IMSI: 208990100001189 (PLMN 208/99), same Key/OPc as the test subscribers
- DB creds (from AMF config): user test / pass test, root pass linux, database oai_db
- Subscriber table in this deployment: **users** (legacy schema)

Part 1 — Commands in order to run

A. One-time setup of the new UE

```
# 1. Create the UE SIM config (copy an existing one, change IMSI)
cd ~/OAI_Beginners_Experiment/openairinterface5g/ci-scripts/conf_files
cp nrue.uicc.conf nrue.uicc2.conf
#   edit nrue.uicc2.conf -> imsi = "208990100001189" (key/opc/dnn unchanged)

# 2. Add a matching subscriber row to the seed file (placement: see Problem 5)
cd ../yaml_files/5g_rfsimulator
#   in oai_db.sql, right AFTER the last pre-seeded users row (...115), add one
#   INSERT INTO `users` VALUES ('208990100001189', ... same key/opc ... );

# 3. Add a new UE service (oai-nr-uenew) in docker-compose.yaml by copying
#   the oai-nr-ue block; point it at nrue.uicc2.conf and
#   set --uicc0.imsi 208990100001189 (see Problem 1: do NOT truncate options)
```

B. Bring the system up — ALWAYS in this order

```
cd ~/OAI_Beginners_Experiment/openairinterface5g/ci-scripts/yaml_files/5g_rfsimulator

# 1. CORE FIRST
docker compose up -d mysql oai-amf oai-smf oai-upf oai-ext-dn

# 2. WAIT until BOTH mysql and oai-amf are (healthy) – do NOT use a fixed sleep.
docker compose ps          # re-run until healthy, OR block automatically:
until [ "$(docker inspect -f '{{.State.Health.Status}}' rfsim5g-mysql)" = healthy ] \
    && [ "$(docker inspect -f '{{.State.Health.Status}}' rfsim5g-oai-amf)" = healthy ]; do
    echo "waiting for mysql + amf..."; sleep 5
done

# 3. gNB, confirm it connects to the AMF
docker compose up -d oai-gnb && sleep 10
docker logs rfsim5g-oai-amf 2>&1 | grep -A6 "gNBs"          # expect "Connected"

# 4. UE last
docker compose up -d oai-nr-uenew && sleep 20
docker logs rfsim5g-oai-amf 2>&1 | grep -iE "208990100001189|authentic|reject" | tail -30
```

C. Verify the UE works

```
# registered? -> expect 5GMM-REGISTERED for 208990100001189
docker logs rfsim5g-oai-amf 2>&1 | grep "208990100001189" | tail -5

# got an IP? -> expect inet 12.1.1.x
docker exec -it rfsim5g-oai-nr-uenew ip addr show oaitun_ue1

# data path works? -> expect 0% packet loss
EXTDN=$(docker inspect -f '{{range .NetworkSettings.Networks}}{{.IPAddress}} {{end}}' \
    rfsim5g-oai-ext-dn | tr ' ' '\n' | grep '192.168.72')
docker exec -it rfsim5g-oai-nr-uenew ping -I oaitun_ue1 -c 5 "$EXTDN"
```

D. Verify / fix the DB row when needed

```
# check the row exists with correct key/opc
docker exec -i rfsim5g-mysql mysql -u test -ptest oai_db \
    -e "SELECT imsi, HEX(`key`), HEX(opc) FROM users WHERE imsi='208990100001189';"

# reliable live insert: clone a known-good row and change ONLY the IMSI
docker exec -i rfsim5g-mysql mysql -u test -ptest oai_db <<'SQL'
DELETE FROM users WHERE imsi='208990100001189';
CREATE TEMPORARY TABLE tmp_u AS SELECT * FROM users WHERE imsi='208990100001100';
UPDATE tmp_u SET imsi='208990100001189';
INSERT INTO users SELECT * FROM tmp_u;
SQL
```

Part 2 — Problems we hit (cause → fix / check)

Problem 1 — UE container exits immediately with code 1

Symptom: log shows [CONFIG] unknown option: --log_config.global_log_opt> then Main child exited normally (with status '1'). No registration attempt at all.

Cause: when copying the USE_ADDITIONAL_OPTIONS line into the new UE service in docker-compose.yaml, the long line got truncated (ended in ...global_log_opt> with a stray >). The UE rejects the unknown option at startup, before touching the radio.

Fix: make the new service's USE_ADDITIONAL_OPTIONS identical to the working oai-nr-ue service, changing only the IMSI (option must end fully, e.g. ... --log_config.global_log_options level nocolor time). Or delete the broken fragment (it's cosmetic). Then:

```
docker compose up -d --force-recreate oai-nr-uenew
docker compose logs -f oai-nr-uenew
```

Problem 2 — Registration Reject with NO authentication step

Symptom: AMF goes 5GMM-DEREGISTERED → COMM-PROC-INIT → REGISTRATION_REJECT_SENT in ~2 ms; no Authentication Request/Response in between.

Cause: the AMF could not obtain authentication vectors — it never challenged the UE. This is a core/DB problem, not a UE problem. (A correct run shows Security Vectors → Auth Request → Auth Response → Security Mode → Accept.)

Check: look for the real reason right before the reject:

```
docker logs rfsim5g-oai-amf 2>&1 | grep -iE "authentic|vector|mysql|db|reject" | tail -20
```

Problem 3 — Can't connect to MySQL server on 'mysql:3306' (111) [THE main issue]

Symptom: [amf_authentication] [error] ... (111) → Cannot connect to MySQL DB → Request Authentication Vectors failure → reject.

Cause: the AMF asked MySQL for vectors BEFORE MySQL was ready to accept connections. Happened every time the UE started too soon — worst right after docker compose down -v, since a fresh volume takes 60–90 s to initialize. docker compose restart (all at once) causes the same race.

Fix: never rely on a fixed sleep; wait for mysql AND oai-amf to be **(healthy)** before starting the gNB/UE (Part 1-B step 2). Order is always **core** → **wait healthy** → **gNB** → **UE**. Avoid restarting the whole stack.

Problem 4 — Reject even though the DB row looks correct

Symptom: Receive information from MySQL with IMSI 208990100001189 appears (row was read), but the UE still ends 5GMM-DEREGISTERED.

Cause: to remove all doubt about the subscriber data, we cloned a known-good row (...100, which registers) and changed only the IMSI, so Key/OPc/SQN were byte-for-byte identical to a working UE. It still failed — proving the subscriber data was never the problem and pointing back to the MySQL timing race (Problem 3).

Check the clone: both rows must show identical key and opc:

```
docker exec -i rfsim5g-mysql mysql -u test -ptest oai_db \
-e "SELECT imsi, HEX(`key`), HEX(opc) FROM users
WHERE imsi IN ('208990100001100','208990100001189');"
```

```
# both -> key FEC86...B8F   opc C42449...7CC1
```

Problem 5 — Edited oai_db.sql but the change doesn't take effect

Symptom: added the INSERT to oai_db.sql, but the running DB doesn't reflect it (or the row sits fragilely near ALTER TABLE ... ENABLE KEYS).

Cause: (a) editing oai_db.sql only re-seeds MySQL if the volume is wiped (docker compose down -v); a plain restart keeps old data. (b) the line was placed at the end of the dump block instead of within the users insert section.

Fix: place the new row right after the last pre-seeded users row and remove the stray ; after the comment. Find the anchor, then re-seed with a full reset:

```
grep -n "208990100001115" oai_db.sql    # e.g. line 209 – put INSERT right after it
docker compose down -v                # then bring up staged (Part 1-B)
# quick live add without reset: use the clone command in Part 1-D
```

Problem 6 — -bash: PW: No such file or directory when checking the DB

Symptom: the command literally contained -p<PW>; the shell tried to run PW.

Cause: placeholder was not replaced with the real password.

Fix: use -ptest (user test) with no space after -p:

```
docker exec -i rfsim5g-mysql mysql -u test -ptest oai_db \
-e "SELECT imsi FROM users WHERE imsi='208990100001189';"
```

Problem 7 — UE-to-UE ping shows 100% loss (but the network was fine)

Symptom: cross-UE pings kept returning 100% loss, or a "0% loss" that was actually a self-ping (from 12.1.1.2 to 12.1.1.2, RTT ~0.02 ms).

Cause: the decisive one: the OAI UE image has NO ping installed by default — and it must be installed in BOTH UEs. Two more testing traps compounded it: (b) shell variables like \$UE_A were empty (set in a different shell, or named UE1 vs UE_A) so ping got no address — "Name or service not known"; (c) pinging the pinger's OWN IP loopbacks internally and never crosses the radio. None of these was a routing fault.

Fix: install tools in the container, use **literal IPs** (or set the vars in the same shell), and always ping the **other** UE's address. A genuine cross-UE result shows two different IPs and a few-ms RTT (~15–18 ms here), never ~0.02 ms.

```
# install ping in BOTH UE containers (both ends)
docker exec -it -u root rfsim5g-oai-nr-ue    bash -lc 'apt-get update -qq && apt-get install -y -qq
iputils-ping'
docker exec -it -u root rfsim5g-oai-nr-uenew bash -lc 'apt-get update -qq && apt-get install -y -qq
iputils-ping'

# read both IPs, then ping the OTHER UE by literal address
docker exec rfsim5g-oai-nr-ue    ip -4 -o addr show oaitun_ue1    # e.g. 12.1.1.2
docker exec rfsim5g-oai-nr-uenew ip -4 -o addr show oaitun_ue1    # e.g. 12.1.1.3
docker exec -it rfsim5g-oai-nr-ue ping -I oaitun_ue1 -c 5 12.1.1.3
```

Part 3 — UE-to-UE communication

Goal: make the two registered UEs (12.1.1.2 and 12.1.1.3) talk to each other over the 5G data plane — through the gNB and UPF.

The real cause: the test tool was missing — not the network

Every "100% packet loss" during this exercise was a **testing artifact, not a routing failure**. The OAI UE image ships without ping, so the tests were failing for lack of a tool. The decisive fix was installing `iputils-ping` in **both** UE containers. Once both had it, UE-to-UE worked immediately, both directions, with **no core-config change at all**. The 5G data plane supported UE-to-UE the whole time.

Working configuration (stock — no change needed)

UE-to-UE worked on the **default** datapath and SNAT settings. No datapath or SNAT change was required:

```
upf:
  support_features:
    enable_bpf_datapath: no      # simple-switch datapath (NOT eBPF)
    enable_snat: yes            # SNAT on
```

What actually made it work

```
# install ping in BOTH UE containers (both ends matter)
docker exec -it -u root rfsim5g-oai-nr-ue    bash -lc 'apt-get update -qq && apt-get install -y -qq iputils-ping'
docker exec -it -u root rfsim5g-oai-nr-uenew bash -lc 'apt-get update -qq && apt-get install -y -qq iputils-ping'

# read both IPs, then ping the OTHER UE by literal address (see Problem 7)
docker exec rfsim5g-oai-nr-ue    ip -4 -o addr show oaitun_ue1    # 12.1.1.2
docker exec rfsim5g-oai-nr-uenew ip -4 -o addr show oaitun_ue1    # 12.1.1.3
docker exec -it rfsim5g-oai-nr-ue    ping -I oaitun_ue1 -c 5 12.1.1.3
docker exec -it rfsim5g-oai-nr-uenew ping -I oaitun_ue1 -c 5 12.1.1.2
```

The result (success)

- UE1 → UE2: from 12.1.1.2 -> 12.1.1.3, 0% loss, ~16 ms avg
- UE2 → UE1: from 12.1.1.3 -> 12.1.1.2, 0% loss, ~16 ms avg

The **~15–20 ms RTT** (not microseconds) is the proof it's real: packets crossed the rfsim radio and the core. Two **different** addresses each way confirms it's not a self-ping.

Dead ends we chased (recorded so you don't repeat them)

- Enabling the eBPF datapath (`enable_bpf_datapath: yes`): irrelevant — UE-to-UE works with eBPF **off**.
- "Disable SNAT" advice (`enable_snat: no`): also not the lever — it works with SNAT **on**.
- Different DNNs + ext-dn forwarding: never needed.
- Self-pings (a UE's own IP, ~0.02 ms) and empty `$UE_` variables — looked like failures but were test-method errors (Problem 7).

Net: it was **never a configuration problem** — the blocker was simply that ping wasn't installed in the containers.

The path (through the gNB and UPF)

What is proven: the **~16 ms RTT** rules out a local loopback, so the packets genuinely traverse the gNB and UPF: UE1 → gNB → (GTP-U/N3) → UPF → back down (GTP-U/N3) → gNB → UE2. With simple-switch + SNAT on, the UPF handles the forwarding between the two UEs.

To pin down the exact turnaround point (inside the UPF vs. briefly via the N6/ext-dn side), capture during a ping — this wasn't definitively captured, but it's the one-line way to confirm:

```
sudo tcpdump -n -i rfsim5g-public udp port 2152 # gNB<->UPF GTP-U: expect request + reply
sudo tcpdump -n -i rfsim5g-traffic # N6/ext-dn side: traffic here = via N6
```

The lessons to remember

1. Most registration failures were one root cause: the AMF needed MySQL before MySQL was ready. The reliable routine, every session:

```
core up → WAIT for mysql + amf (healthy) → gNB up → UE up → verify
```

2. Before trusting a "100% loss" result, verify your test tools and method: install ping/tcpdump in the container, use **literal IPs**, and make sure you're pinging the **OTHER** UE (different address, few-ms RTT).

3. UE-to-UE worked on the **stock config** (simple-switch, enable_bpf_datapath: no, enable_snat: yes) — no datapath or SNAT change was needed. The blocker was tooling (ping missing in the UEs), not routing. Don't chase config levers before verifying the test tool exists on both ends.

Success looks like: AMF shows 5GMM-REGISTERED for the IMSI, the UE has oaitun_ue1 with a 12.1.1.x IP, ping to the ext-dn is 0% loss, and ping to the **other UE** (with ping installed in both) is 0% loss at a few-ms RTT.